📄 Rapport technique du projet : Prédiction du prix des ordinateurs portables

🔧 1. Contexte et objectifs

Dans le contexte de l'essor du commerce en ligne et de l'informatique décisionnelle, nous avons conçu une **application intelligente** capable d'**estimer automatiquement le prix d'un ordinateur portable** à partir de ses caractéristiques techniques.
L'objectif principal est de **fournir une solution prédictive rapide et précise**, qui pourra servir d'outil d'aide à la décision pour les vendeurs, distributeurs ou analystes du marché.

🧪 2. Étapes du projet

📊 2.1. Analyse exploratoire des données (EDA)

Nous avons commencé par charger un jeu de données contenant plusieurs attributs techniques et commerciaux des laptops :

- Marques (Company, Brand)
- Composants (RAM, CPU, GPU, disque)
- Détails de l'écran (résolution, technologie tactile, IPS)
- Autres : système d'exploitation, poids, etc.

Démarches entreprises :

- Vérification des valeurs manquantes et nettoyage.
- Création de colonnes dérivées utiles :
  - `Touchscreen` et `IPS` extraits depuis les chaînes de `ScreenResolution`
  - `X_res`, `Y_res`, puis `ppi` (densité de pixels) calculés via les formules.
  - Détection et encodage de colonnes catégorielles.

Cela nous a permis d'améliorer la **qualité du dataset** avant la modélisation.

⚙️ 2.2. Prétraitement et encodage

Nous avons encodé manuellement les colonnes catégorielles via `LabelEncoder`, en conservant les encodeurs dans un dictionnaire pour pouvoir les réutiliser dans l'interface utilisateur.

Nous avons également mis en œuvre un **scaling (normalisation)** via `StandardScaler` pour améliorer la performance du modèle. Le scaler a été **sauvegardé dans un fichier `scaler.pkl`** pour être rechargé dans l'application.

🤖 3. Modélisation

🔍 3.1. Choix du modèle

Le **Random Forest Regressor** de `scikit-learn` a été choisi car :

- Il gère bien les données tabulaires hétérogènes.
- Il réduit le surapprentissage via l'agrégation de plusieurs arbres.
- Il nécessite peu de mise à l'échelle des données (mais nous l'avons fait par sécurité).

⚙️ 3.2. Entraînement

- Split en `train/test` à 80/20.
- `model.fit(X_train, y_train)`
- Sauvegarde du modèle via `joblib.dump(model, "laptop_price_model.pkl")`

Les performances ont été jugées **satisfaisantes en MSE et R^2**, suffisantes pour une application de prédiction grand public.

🖼️ 4. Interface utilisateur (Streamlit)

🛠️ 4.1. Technologie choisie

Nous avons utilisé **Streamlit**, pour les raisons suivantes :

- Déploiement rapide et léger d'applications ML.
- Interface claire, ergonomique et facile à personnaliser.
- Intégration transparente avec `joblib`, `numpy` et `scikit-learn`.

📥 4.2. Entrées utilisateur

L'utilisateur renseigne via l'interface :

- Des **valeurs catégorielles** : marque, processeur, mémoire, système d'exploitation, etc.
- Des **valeurs numériques** : poids, RAM, taille écran, résolution, etc.
- Des **options binaires** : tactile, IPS.

Ces champs sont automatiquement encodés, combinés, transformés et passés au modèle pour prédiction.

💡 4.3. Sortie

L'application retourne :

- Une **estimation du prix en FCFA**.
- Une **interface dynamique, simple et compréhensible** même pour un utilisateur non technique.

💾 5. Fichiers générés et utilisés

Fichier	Description
`laptop_price_model.pkl`	Modèle Random Forest entraîné
`encoders.pkl`	Dictionnaire des encodeurs `LabelEncoder`
`scaler.pkl`	StandardScaler utilisé pour le preprocessing
`app.py`	Code source de l'application Streamlit

✅ 6. Résultats et avantages

- Interface simple, intuitive et ergonomique.
- Temps de réponse rapide.
- Résultats fiables dans une marge d'erreur acceptable.
- Modèle facilement améliorable si de nouvelles données sont ajoutées.

📈 7. Pistes d'amélioration

- Ajouter une visualisation de l'évolution des prix par marque ou configuration.
- Intégrer un retour utilisateur pour affiner les prédictions.
- Déployer l'application en ligne (Streamlit Cloud ou serveur local).

🔚 Conclusion

Ce projet démontre la **possibilité concrète d'utiliser l'intelligence artificielle appliquée au pricing**, avec une approche accessible et portable. Grâce à l'utilisation judicieuse de bibliothèques robustes (`pandas`, `scikit-learn`, `joblib`, `streamlit`), nous avons pu réaliser un outil complet, prêt à l'usage, en conservant une architecture flexible pour d'éventuels perfectionnements.

Par Magloire AFFOKPE